



Task Offloading in Vehicular Edge Computing Network Using Deep Reinforcement Learning

Shailja Kumari^{1*} Divya Gupta¹

¹Department of Computer Science and Engineering, Chandigarh University, Mohali, 140413, Punjab, India

* Corresponding author's Email: spsk1202@gmail.com

Abstract: The advancement of intelligent vehicular systems has fostered a safer and more comfortable environment for both drivers and passengers. Emerging applications in autonomous driving, infotainment, and real-time traffic management hold significant potential to enhance users' daily mobility experiences. However, deploying computationally intensive applications directly on vehicles remains a challenge due to their limited processing and energy resources. This study proposes a Task Offloading using Deep Reinforcement Learning (DRL) for Vehicular Edge Computing Network (VECN), to optimize computational resource allocation under dynamic vehicular conditions. The proposed approach presents a novel integration of enhanced Deep Q-Network (DQN) architecture with priority-aware offloading and adaptive reward tuning, enabling efficient decision-making. Proposed approach incorporates task urgency, vehicular task density variation, and energy constraints into its state representation and dynamically balances using the reward function. Using the Google Cluster Dataset, the performance evaluated against Local Processing, DQN, and Q-learning. Results show that proposed approach reduces task completion delay by up to 39.4%, improves energy efficiency by 31.2%, and decreases task failure rates by over 70% compared to local processing. These improvements demonstrate proposed approach's effectiveness in VECN.

Keywords: Task offloading, Deep reinforcement learning, Deep Q network, Smart transportation, Internet of vehicle, Vehicular edge computing.

1. Introduction

As the Internet of Vehicles (IoV) continues to grow quickly, many applications that require a lot of processing and are sensitive to delays have been introduced to enhance human-vehicle interaction and overall driving experience [1]. Leveraging IoT-enabled vehicular communication, vehicles can now exchange data with roadside infrastructure such as traffic lights, sensors, and other vehicles [2]. This interaction not only improves safety and traffic efficiency but also provides drivers with real-time information, enabling vehicles to disseminate alerts regarding traffic congestion, accidents, or hazardous road conditions to nearby vehicles. These capabilities significantly reduce travel time and fuel consumption while enhancing road safety [3].

Among the many emerging applications in intelligent IoV systems, autonomous driving stands

out as a key technology for improving trip efficiency and safety [4]. However, as the quantity of linked vehicles keeps increasing, efficient communication management becomes increasingly critical to ensure system performance and reliability [5]. To support autonomous driving, vehicles must use on-board cameras and sensors to perceive their surroundings in real time. The acquired image data must be processed by on-board units (OBUs), which extract relevant features to support decision-making. However, processing large volumes of high-resolution data within strict time constraints poses significant challenges. In particular, high latency is a critical concern, and as data complexity increases, the computational capacity of OBUs may become insufficient to meet real-time task execution requirements. Researchers initially proposed the Cloud Computing (CC) paradigm as a solution for computationally demanding tasks, leveraging centralized cloud servers equipped with vast storage

and processing capabilities [6]. While CC offers high-performance computing, it still faces challenges in meeting the real-time responsiveness required by vehicular applications due to its inherent communication delays. The processing load on vehicles can be reduced by offloading computing task to cloud servers [7]; however, the transmission of large volumes of data introduces unstable connections and significant transmission delays, especially in highly mobile environments [8]. To address these limitations, Mobile Edge Computing (MEC) has emerged as a promising alternative. By deploying computing nodes closer to end users, MEC substantially reduces communication latency compared to CC, and its effectiveness, increases as the number of connected vehicles grows [9]. MEC enhances the performance of VECNs by compensating for the drawbacks of cloud-based architectures [10]. MEC is a relatively recent paradigm that enables low-latency computing services for vehicular networks by placing Edge Servers (ESs) near the network edge. Despite its advantages, MEC faces critical challenges in VECN environments. Due to the dynamic and high mobility nature of vehicular networks, ESs often receives a large number of offloading requests simultaneously. This leads to fluctuating availability of computational resources, especially under uneven vehicle density distributions [11]. As task loads increase, resource-constrained ESs may struggle to process all incoming tasks efficiently.

Furthermore, traditional networking and task scheduling techniques, which are often time-consuming and energy-intensive, may become ineffective under such dynamic and resource-limited conditions [12]. Therefore, designing a scalable and adaptive resource management strategy is crucial to fully exploit the benefits of MEC in vehicular edge environments. To address the aforementioned challenges, computational tasks can be partially offloaded from Roadside Units (RSUs) to Cloud Servers (CSs). This paper investigates a hybrid architecture where MEC and CC collaborate within a VECN to manage tasks with stringent performance requirements [13]. A DRL model is employed to learn the mapping between input parameters and optimal offloading strategies, enabling more effective decision-making for resource allocation and task distribution [14].

Among various machine learning approaches, DRL has emerged as a promising technique capable of outperforming traditional methods. While deep learning has seen significant progress in domains such as natural language processing, pattern recognition, and image processing, it is increasingly

being applied to time-sensitive and computationally intensive scenarios, such as real-time vehicle to infrastructure navigation and smart driving. For example, Li et al. [15] applied a Deep Q-Network (DQN) to explore partial task offloading in MEC networks with multi-user. However, their work highlighted the challenge of coordination between cloud and edge servers, leading to increased system costs as the number of users grows. Although machine learning applications in VECNs are expanding, they are still in their early stages. Some studies employ deep learning and convolutional neural networks (CNNs) for traffic flow prediction, yet the application of DRL in vehicular task offloading remains limited. This gap motivates the need to explore DRL for optimal offloading decisions in VECNs. However, heterogeneous task demand is critical challenge to design an effective DRL-based task offloading system.

In the present research, we propose an optimisation framework based on DRL that tackles these challenges by dynamically managing task scheduling and resource allocation in VECNs. The proposed method learns to make optimal offloading decisions under varying network conditions, aiming to minimize latency, reduce energy consumption and maximize overall system utility. The task offloading problem in this study is formulated as a DRL problem comprising four essential components: agents, environmental states, action space, and reward function. The following is a summary of the main contributions of this work:

1. **Three-Layer VECN Architecture with MEC and CC Integration:** We propose a three-layer VECN architecture that integrates both MEC and CC. The dynamic vehicular scenario involves autonomous vehicles traveling along roadways in the vehicle layer, where RSUs receive task requests. These tasks are processed by ESs, while Base Stations (BSs) manage load balancing among ESs by coordinating with RSUs. Additionally, a CS in the upper layer handles computation-intensive tasks and supports the DRL agent for task offloading decisions. We also consider binary task offloading decisions to improve adaptability and flexibility in the offloading mechanism. An optimized strategy is presented for cooperative resource allocation and task offloading with the goal of reducing task processing costs while meeting delay and energy consumption constraints.
2. **Improved DQN for Enhanced Stability:** To overcome the overestimation problem inherent

in standard Q-learning algorithms, we design an enhanced Deep Q-Network (DQN) that improves system performance and stability. Extensive simulations are conducted to compare the proposed method with conventional approaches, including Q-learning, standard DQN, and local task processing.

3. Evaluation of Performance: Results from simulations show that the proposed DRL-based offloading system significantly outperforms existing methods in terms of processing delay, energy efficiency, and task failure rate, validating the effectiveness of our model.

This article's remaining content is organised as follows: The pertinent research is reviewed in Section 2. The objective function is developed in Section 3 with the system model. The suggested algorithm for an optimal task offloading system and a DRL-based algorithm in a vehicular network are demonstrated in Section 4. Section 5 presents performance evaluations, while Section 6 concludes up the entire procedure.

2. Related work

Integrating Edge Computing (EC) with Cloud Computing (CC) can substantially reduce the computational burden on mobile and vehicular devices, making it possible to carry out computationally demanding and data-driven applications in real time. As a result, the modelling and optimization of offloading task and resource allocation have been widely explored in existing literature. In [16], the authors addressed the problem of multiple user task offloading in an end edge-cloud system, where multiple user devices compete for limited communication and computational resources. To solve this challenge, they proposed a decentralized task offloading method using game theory, which determines the Nash equilibrium for optimal offloading strategies among users. However neglect critical aspects such as priority-aware resource allocation policies, and absence of real-world workload integration, joint load balancing and real-time trade-offs between delay, energy consumption, and task success rate. Similarly, in [17], the authors investigated cost-efficiency optimization in MEC enabled smart vehicular networks, focusing on three key aspects: virtual machine deployment, task scheduling, and base station association. They formulated the optimization objective as a Mixed-Integer Nonlinear Programming (MINLP) problem and subsequently linearized it into a Mixed-Integer Linear Programming (MILP) model to minimize overall

system costs. In another study, [18] proposed a strategy combining offloading task and resource allocation to lower overall task processing latency for all vehicles. Their method included channel assignment, task scheduling, and computing resource distribution between vehicles and RSUs. This approach accounted for vehicle classification, task diversity, and the flexibility of task execution. However, it lacked consideration of vehicle mobility, a key challenge in real-world vehicular networks. Unlike prior studies, our work integrates context-aware factors such as user preferences and traffic density, which play a crucial role in dynamic vehicular environments. To manage computational resources effectively and prioritize time-sensitive tasks, we employ an enhanced Deep Q-Network based offloading mechanism that dynamically adapts to environmental changes and improves overall system performance.

According to [19], Vehicles can act as mobile edge servers, providing adjacent devices with computing services. Maximising the long-term usefulness of IoV was the optimisation goal in that study. Nevertheless, a lot of earlier research depended on binary task offloading decision-making, which, despite being computationally efficient, are often too simplistic to derive optimal policies in large-scale systems characterized by complex interactions between cloud and edge infrastructures. To address this limitation, [20] proposed a partial computation offloading strategy, there are two categories for vehicular tasks: one processed locally and the other offloaded to the edge. This hybrid approach improves vehicular performance by balancing latency and computational load. In another study [21], the requested video content was either retrieved from the Internet or fetched from base station caches. The authors proposed an integrated resource allocation strategy for connected vehicles by jointly optimizing networking, computing, and caching resources. However, considering all three dimensions simultaneously significantly increases problem complexity. To solve the resulting optimization challenge, a DRL-based method was adopted. A notable contribution in [22] examined computational task scheduling in typical vehicular environments based on edge computing. The authors combined a Convolutional Neural Network with a DRL model to determine optimal offloading policies.

Further, [23] introduced a computation migration mechanism, enabling offloaded tasks to be executed either locally by MEC servers or remotely by central cloud servers. To reduce energy consumption and task processing latency, the

authors formulated a complex optimization problem and solved it using a two-stage heuristic algorithm. This method involved an initial decision-making step based on randomization and semi definite relaxation to determine offloading candidates, followed by optimization of resource allocation for the selected users. In the context of smart cities, [24] proposed a DRL-based caching strategy to reduce content duplication in vehicular networks. Here, an agent learns to select optimal actions by observing state information from MEC servers and base stations, thereby achieving efficient resource allocation. Moreover, [25] presented a paradigm for multiple user, non-cooperative computation offloading in VECNs that is based on game theory. This approach aims to regulate each vehicle's task offloading frequency by formulating a reward function that accounts for communication models, application demands, inter-vehicle competition, and distances to MEC nodes. A decentralized best-response strategy was developed and shown to achieve a stable and ideal balance. However, the study does not adequately address the difficulties presented by high-mobility situations because it mostly concentrates on low-mobility or static circumstances, which are intrinsic to vehicular networks. A combined challenge of resource allocation and RSU selection was investigated in [26], where the primary objective was to minimize overall task offloading delay under constraints of computing resources and bandwidth availability.

To support distributed learning in mobile small-cell networks, [27] proposed a federated learning framework based on Gated Recurrent Units (GRUs), enabling model training without transferring raw data across nodes. However existing study incorporated synthetic data and not considered trade-offs between delay, energy consumption, and task success rate. In [28], a hybrid transmission and offloading strategy was introduced, allowing fog nodes to partially compute tasks locally and partially offload them to other idle fog nodes. This approach acknowledges the dynamic nature of vehicular traffic and the fluctuating availability of edge resources in IoV environments. For example, users prefer local processing when edge servers are congested and task offloading in the presence of sufficient processing resources. However, many prior works fail to fully address such dynamic and stochastic network conditions. The server selection problem in a system comprising multiple users, central cloud servers, and edge nodes was addressed in [29]. The authors proposed a heuristic scheduling strategy aimed at maximizing the offloading success probability. To enhance Quality of Service (QoS), a

threshold-based method was developed, leveraging the low latency of local edge resources and the computational abundance of Internet cloud servers. However, this work does not incorporate resource management or account for real-time network dynamics, limiting its practical applicability. In [30], the authors tackled joint edge computing and cache resource allocation using the "Deep Deterministic Policy Gradient" (DDPG) algorithm. Key components of this model include experience replay memory, which stores and reuses network interactions to improve learning stability and efficiency.

The task offloading mechanism presented in [31] enables vehicles to dynamically decide whether to process tasks locally or offload them to neighbouring vehicles, edge servers, or the cloud. The system aims to minimize delay and cost by integrating DRL with Graph Convolutional Networks (GCNs), thereby enhancing the agent's network representation and decision-making accuracy. The combination of Double Deep Q-Networks (DDQN) and GCNs further strengthens policy learning and system performance. However, existing studies have not accounted for dynamic variations in vehicular task density and computational resource availability, thereby limiting their applicability in highly dynamic vehicular environments. In a multi-user vehicular scenario involving multiple servers, [20] considered both load balancing and task offloading for effective resource allocation. This framework incorporates vehicular mobility by enabling vehicles to choose their preferred edge servers on their own, depending on the demands of the task at present and server status, thereby improving system responsiveness and adaptability. In contrast to previous studies, this paper investigates the collaborative integration of CC and MEC within VECNs, aiming to identify a near-optimal task offloading strategy based on a variable CC/MEC distribution ratio. To improve the flexibility and responsiveness of the offloading policy in dynamic environments, we adopt a binary offloading model that allows flexible decision-making between local, edge, and cloud execution.

Furthermore, our approach comprehensively accounts for the dynamic characteristics of both the communication and computing subsystems, including randomized vehicular traffic patterns and the time-varying computational capacity of edge servers. To increase our DRL algorithm's training stability and mitigate the risk of Q-value overestimation without introducing significant computational overhead, we incorporate separate

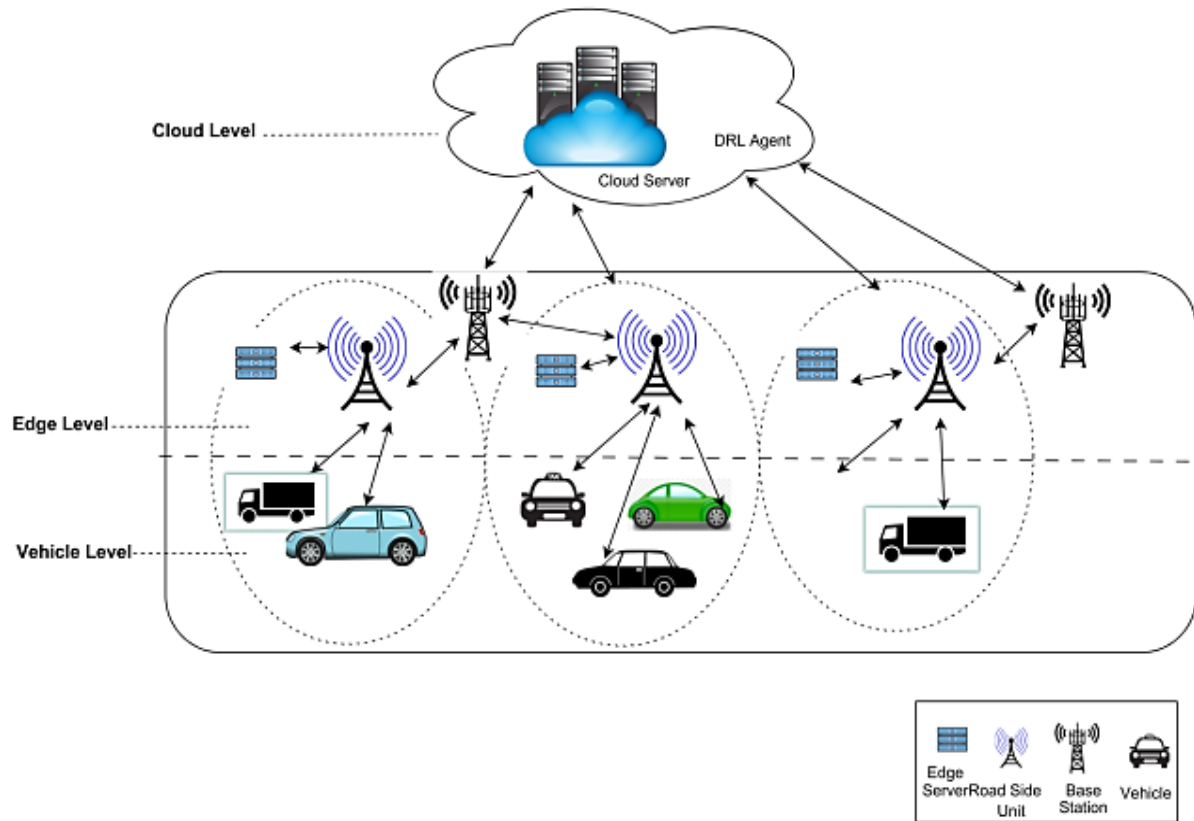


Figure. 1 System Architecture

target and primary networks into the DRL framework

3. The system model and formulation of problem

The detailed system model, comprising the network, mobility, task, computational, and communication models, is proposed in this part. Based on streets or other criteria, a city's vehicle network might be divided into several zones as seen in Fig. 1. A central Cloud server with plenty of computational resources is located at cloud level. We assume that cellular networks can completely cover metropolitan areas, much like in References [32]. Additionally, within each zone, a number of RSUs are placed along the base station. Note that MEC servers are installed on RSUs. Vehicles ensure high-quality communications within a short range by uploading tasks to RSUs. RSUs communicate with one another via Base Stations (BS) and receive global data on tasks related to vehicle offloading. BS must act as the traffic management center if RSUs only communicate with vehicles and BS, which would result in a significant communication and processing load. We just look at one metropolitan zone for

Table 1. Symbol representation and description

Notation	Description
T_m^t	Task at vehicle m during time slot t
$T_{(m,n)}^t$	Task offloaded from vehicle m to n ES
$z(T_m^t)$	Decision of task Offloading at vehicle m
G	Set of Vehicle in a Zone
H	Set of Edge Server
EP	Edge level processing
D_i^t	task's Size in bits from vehicle i at time t
CP	Cloud level processing
W_m^{exec}, W_n^{exec}	Execution power of vehicle m , EP n
W_{CP}^{exec}	Execution power of CP
f_m^{VP}, f_n^{EP}	Execution capacity of vehicle m , EP n
f_m^{CP}	Execution capacity of CP
$W_{(m,n)}^{comm}$	Transmission power of vehicle m to n
C_m^t	Execution resource required per bit
$C_{m,t}^{VP}, C_{m,t}^{EP}$	Total cost at vehicle, EP
$C_{m,t}^{CP}$	Total cost at CP
$t_{m,exec}^{VP}, t_{m,n}^{EP}$	Total time at vehicle, EP
$t_{m,t}^{CP}$	Total time at CP
$E_{m,t}^{VP}, E_{m,n}^{EP}$	Total energy consumption at vehicle, EP
$E_{m,t}^{CP}$	Total energy consumption at CP

simplicity, but the model is easily adaptable to other zones. The main notation of the symbol with description is represented by Table 1. By taking into account variables such as device capacity, task

transmission rate, queue waiting delay, processing delay, task tolerance delay, and task computation energy consumption, this model aims to make informed offloading decisions. Additionally, the goal is to reduce server overload and network congestion, which can decrease the cost of computation and task drop rates.

The proposed work includes a number of models, including computing, communication, and a task with an offloading decision model.

3.1 Communication model

Transmission delays and network bandwidth consumption occur when the vehicle transfers the task to the edge server. It is necessary to maintain the bandwidth consumed for task offloading during time slot t below the maximum bandwidth capacity. Reducing the total task cost generated by all phases is the main objective of the solution. A wireless communication link is needed to transfer work from the vehicle to the cloud centre or EP. The data transmission rate, represented by the symbol R_m , is established via Shannon's formula in Eq. (1):

$$R_m = B_m \log_2 \left(1 + \frac{(W^{comm} * h)}{N} \right) \quad (1)$$

Where B_m the user-allocated is channel bandwidth, W^{comm} symbolises the vehicle's transmission's power, N represents the Gaussian white noise, and h represents the channel gain.

3.2 Task model

We assume that for a given time slot t , a vehicle may generate a maximum of one task. The tuple $\langle D_m^t, \theta^t, C_m^t \rangle$ represents a task T_m^t produced during time slot t by vehicle m . The tuple includes the task size in bits, the task deadline, and the required computational resource, respectively.

The decision model provides the offloading decision if vehicle m receives a task T_m^t at time slot t by decision variable $z(T_m^t) \in \{-1, 0, 1\}$ and expressed by Eq. (2).

$$z(T_m^t) = \begin{cases} 0 & \text{Execution by vehicle } m \\ 1 & \text{Execution by EP } n \\ -1 & \text{Execution by CP} \end{cases} \quad (2)$$

This model considers binary offloading, demonstrating that a full task is either offloaded or not. The task cannot be broken down into smaller tasks.

3.3 Mobility model

The vehicle motion is illustrated with the assumption that, despite moving randomly and shifting locations regularly, vehicles' placements only little alters over brief periods of time. The way a vehicle moves might be seen as a distinct image that moves in discrete time intervals. Because they are mobile, vehicles represented by $v_j(t)$ create computation-intensive tasks. Position $P_j(t)$ and velocity $v_j(t)$ over time t indicate the vehicle's particular mobility pattern represented by Eq. (3).

$$P_j(t) = \int_0^t v_j(t) dt \quad (3)$$

3.4 Load balancing model

The system established to avoid overburden. To distribute the load evenly among all servers, tasks are allocated based on edge server E_j that has the load L_j . Eq. (4) shows how much load Edge server j is under and comparable to references [33].

$$L_j(t) = \sum_{i=1}^m \sum_{j \in n} \frac{D_{ij}}{W_n^{exec}} z(T_m^t) \quad (4)$$

Where $z(T_m^t) = 1$, for each task that the vehicle transfers to the edge server. The vehicle i generates the group of tasks, and the edge server (E_j) is notified whether a task is offloaded. Load on ES must be less than or equal to the edge server's maximum load capacity.

3.5 Computation model

3.5.1. Vehicle level processing

The delay for local processing $t_{m,exec}^{VP}$ of task T_m^t , produced during time slot t by vehicle m , can be evaluated when the vehicle performs the task request itself as follows by Eq. (5).

$$t_{m,exec}^{VP} = \frac{D_m^t * C_m^t}{f_m^{VP}} \quad (5)$$

At this moment, $E_{m,t}^{VP}$ represents and determines the amount of energy consumed for local processing of task T_m^t in Eq. (6).

$$E_{m,t}^{VP} = t_{m,exec}^{VP} * W_m^{exec} \quad (6)$$

Where W_m^{exec} represents vehicle m 's processing capacity during time slot t in (J/s).

Hence, Eq. (7), which represents the cost of local task processing, can be considered as the sum of Eq. 5 and Eq. (6).

$$C_{m,t}^{VP} = \delta t_{m,exec}^{VP} + (1 - \delta) E_{m,t}^{VP} \quad (7)$$

Where the weighting parameter, δ , has values between $0 < \delta < 1$.

3.5.2. Edge level processing

When the vehicle is unable to process the generated task, the decision model indicates that the task offloaded to EP n if $z(T_m^t) = 1$.

From vehicle m to EP n , the task communication time is represented by $t_{m,n}^{comm}$ in Eq. (8).

$$t_{m,n}^{comm} = \frac{D_m^t}{R_{m,n}} \quad (8)$$

Where $R_{m,n}$ is the rate at which data is transmitted from vehicle m EP n .

Following offloading, $t_{n,exec}^{EP}$ is the processing time in task execution at EP n , indicated by Eq. (9).

$$t_{n,exec}^{EP} = \frac{D_m^t * C_m^t}{f_n^{EP}} \quad (9)$$

So total completion time of task at EP n is $t_{m,n}^{EP}$ and denoted in Eq. (10).

$$t_{m,n}^{EP} = t_{m,n}^{comm} + t_{n,exec}^{EP} \quad (10)$$

Therefore, the energy consumed to complete the task in EP n that vehicle m offloads is $E_{m,n}^{EP}$ and is written as by Eq. (11).

$$E_{m,n}^{EP} = (W_{(m,n)}^{comm} * t_{m,n}^{comm}) + (W_n^{exec} * t_{n,exec}^{EP}) \quad (11)$$

where $W_{(m,n)}^{comm}$ is transmission power of vehicle m to EP n and W_n^{exec} is processing power of EP n for task from vehicle m at time t in (J/s). Eq. (12) indicates the weighted sum of Eq. 10 and Eq. (11), which together describe the entire cost of offloading and processing task T_m^t at EP n .

$$C_{m,t}^{EP} = \delta t_{m,n}^{EP} + (1 - \delta) E_{m,n}^{EP} \quad (12)$$

3.5.3. Cloud level processing

Task processing can be offloaded to the cloud centre based on a decision model when $z(T_m^t) = -1$ if it cannot be completed at the vehicle and EP because of latency limitations or resource

unavailability. Eq. (13) demonstrates the time required to transmit the task from vehicle m to cloud centre CP.

$$t_{m,CP}^{comm} = \frac{D_m^t}{R_{m,CP}} \quad (13)$$

Where $R_{m,CP}$ is the data transfer rate, as calculated by Shannon's theorem, from vehicle m to cloud centre CP. Eq. 14 represents the time consumption for task T_m^t execution in cloud centre CP, which is t_{exec}^{CP} .

$$t_{exec}^{CP} = \frac{D_m^t * C_m^t}{f_n^{CP}} \quad (14)$$

So total completion time of task at CP is $t_{m,t}^{CP}$ and calculated in Eq. (15)

$$t_{m,t}^{CP} = t_{m,CP}^{comm} + t_{exec}^{CP} \quad (15)$$

The energy consumption in completing of task at CP offloaded by vehicle m is $E_{m,t}^{CP}$ and represented by Eq. (16)

$$E_{m,t}^{CP} = (W_{(m,CP)}^{comm} * t_{m,CP}^{comm}) + (W_{CP}^{exec} * t_{exec}^{CP}) \quad (16)$$

Where $W_{(m,CP)}^{comm}$ is transmission power of vehicle m to CP and W_{CP}^{exec} is processing power of CP for task from vehicle m at time slot t in (J/s). Therefore, Eq. (17), which is the weighted sum of Eq. (15) and Eq. (16), can be used to describe the entire cost of offloading and processing task T_m^t at CP.

$$C_{m,t}^{CP} = \delta t_{m,t}^{CP} + (1 - \delta) E_{m,t}^{CP} \quad (17)$$

3.6 Resource management model

Using resource management techniques like load balancing and dynamic resource modification can guarantee high utilisation and avoid overloading edge servers, which can be employed in the environment based on edge computing. To prevent a single server from acting as a bottleneck, load balancing is crucial for allocating tasks evenly among edge servers. By ensuring efficient resource utilisation, the system reduces the rate of server overload by maintaining updates on the edge server's load and shifting tasks as required.

Allocation and Dynamic Adjustment of Resource: Taking into account the edge servers' present workload and the requirements of the incoming

tasks, computational resources like CPU and memory are distributed to them. Let K_j represent the edge server j 's total computational resources. Adapting resource allocations in real time according to task needs and network conditions is known as dynamic resource adjustment. This promotes effective resource use and flexibility in response to shifting circumstances. The patterns of demand and load that exist presently direct the resource modification. Let $K_{j,alloc}$ stand for the resources that are accessible at time t at the edge server j . Eq. (18) provides the dynamic adjustment ($\Delta K_j(t)$):

$$(\Delta K_j(t)) = K_j - L_j(t) \quad (18)$$

Where the server's current load at time t is denoted by $L_j(t)$ and K_j is the edge server j 's total resources. The urgency of a task $T_i^j(t)$ and the available resources affect how much extra is allocated to it. Task $T_i^j(t)$ has an urgency U_i^j , which is given by Eq. (19).

$$U_i^j = \frac{1}{T_{i,j,deadline} - t_{i,j,current}} \quad (19)$$

Where the task's deadline is $T_{i,j,deadline}$ and the current time is $t_{i,j,current}$. When allocating resources, tasks that have higher urgency are given priority. The resources $K_{j,alloc}$ assigned to task $T_i^j(t)$ are modified by the urgency specified by Eq. 20.

$$K_{j,alloc} = K_{j,avail}(t) * U_i^j \quad (20)$$

Thus, the ratio of tasks rejected to all tasks generated by vehicles is used to compute the task offloading failure rate.

Objectives: Optimising task offloading to significantly enhance experience quality and making effective use of edge and cloud resources are the primary objectives. With consideration of factors such as latency, energy consumption, and reducing the failure rate of task offloading, the quality of the experience is enhanced. As a result, we use constraints to establish objectives by function.

Reducing the overall cost which is the weighted sum of the energy and delay is the objective, as shown in Eq. (21). Our objective is to use the derived decision variable $z(T_i)$ to reduce the time-energy trade-off.

$$\min_{z(T_i)} Cost(z) = \min_{z(T_i)} (\beta * Delay(z) + \alpha * Energy(z)) \quad (21)$$

Delay is the total time, and Energy is the total energy for communication, execution during task completion. where β, α are the trade-off weights for system cost between delay and energy consumption. Subject to:

$$\begin{aligned} C1: & z(T_m^t) \in \{-1, 0, 1\} \\ C2: & t_{m,exec}^{VP}, t_{m,n}^{EP}, t_{m,t}^{CP} \leq \theta^t \\ C3: & f_m^{VP} \leq f_{max}^{VP} \\ C4: & f_n^{EP} \leq f_{max}^{EP} \\ C5: & 0 \leq W_{m,n}^{comm} \leq W_{max} \end{aligned}$$

Only one computing node can handle a task produced by vehicle, according to the constraint $c1$; $c2$ indicates that the task's deadline should not be exceeded by the overall delay for task execution by vehicle, EP, or CP;

While $c3 - c4$ indicates that the task's total processing power cannot exceed the node's maximum computing capabilities. $c5$ Indicate that the task's transmission power cannot exceed the maximum transmission power.

4. DRL-based algorithm in vehicular network

The optimisation problem for resource allocation is described as a DRL procedure. It is very difficult to resolve this complex issue using conventional optimization techniques, since variables, the vast universe of environmental conditions, and system actions change dynamically over time. As a result, we effectively and efficiently generate system actions for vehicles by utilizing the recent progressive DQN. In order to lessen the load on ESs, BS is responsible for connecting with the environment. We assume that the compute state of the MEC server is shared among RSUs and updated in real time. The MEC servers and vehicles provide the status to the BS. By developing the system state and selecting the most effective course of action, the BS can decide to offload with the help of the DRL agent based on the real-time mobility of vehicles obtained through broadcasting. Lastly, vehicles can be broadcast all optimal actions.

4.1 DRL in Vehicular Networks

In Reinforcement Learning (RL), the four fundamental components are the agent, environmental states, actions, and rewards. In each

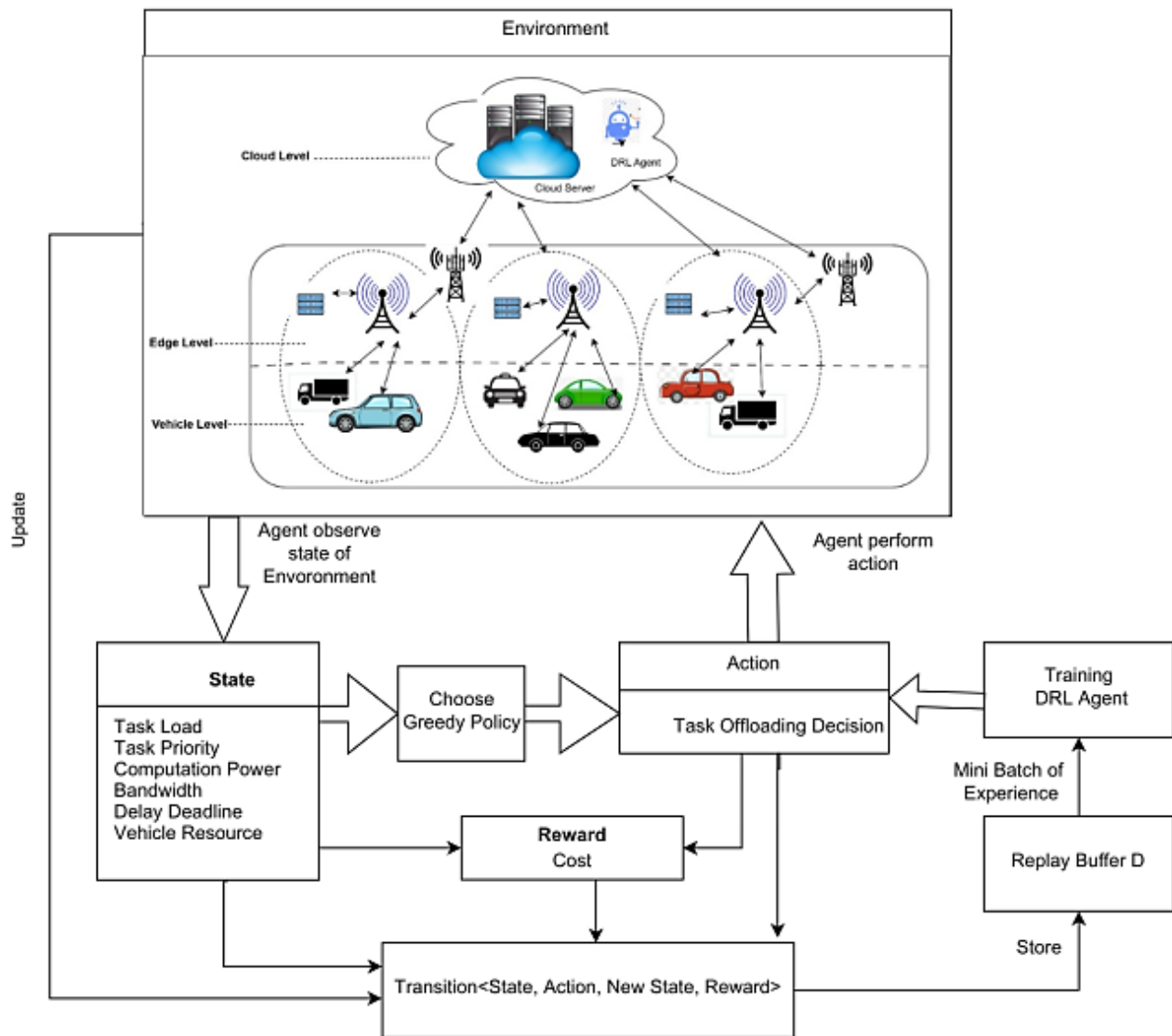


Figure. 2 Proposed architecture of DRL

interaction cycle (episode), the agent determines where to offload and process a task based on the observed state of the environment and receives a reward signal following the execution of the selected action. In order to formulate the RL problem as an optimal control problem, the environment is usually modelled as a Markov Decision Process (MDP). The MDP framework assumes a finite and discrete set of states and actions, and the agent learns in order to optimise its total benefit throughout time by engaging in a sequence of decisions through interaction with the environment [40]. Due to the limited computational and caching capabilities of individual vehicles, deploying computation-intensive deep neural network (DNN) applications on-board is impractical. Unlike traditional supervised learning, RL does not rely on labelled datasets, even if expert knowledge

is available. Instead, it learns via trial-and-error interactions and considers delayed rewards, both of which are central to its learning mechanism. These properties enable the agent to evaluate long-term outcomes and effectively balance the exploration–exploitation trade-off. Notable RL algorithms include Q-learning and DQNs. However, conventional dynamic programming techniques struggle to identify optimal policies due to the high-dimensional state–action space and the incomplete knowledge of state transition probabilities in vehicular networks. To overcome these limitations, this study employs DRL to tackle the server selection and task offloading problem more efficiently.

We adopt the DQN as value-based reinforcement learning strategy for task offloading. By integrating DQN, the centralized agent (or

controller) can learn optimal offloading decisions and efficiently manage the system's dynamic behaviour in real-time. Fig. 2 shows the proposed DRL-based architecture.

In order to perform task offloading based on DRL, the objective function is formulated as MDP. The vehicle that generates the task needs computational resources. The DRL agent in the model decides what to do next based on states and enhances learning through interaction with the environment. In an edge computing-based network, the agent acts in accordance with predicted decisions to reduce cost. The vehicle's attributes, the task's features, the resources at ES, and each additional variable that is essential to task computation are all represented by the state of the environment.

For a task produced at a particular vehicle during time slot t by executing the action, the decision variable z determines the location of task computation. The environment is impacted by action. The agent uses the policy to take action on specified states for state transitions and is rewarded. The agent's objective is to maximise task offloading reward in order to minimise offloading costs. Proposed approach considers multi-objective DRL for dynamically balancing competing objectives in the reward formulation. The reward function incorporate priority, energy consumption, delay, and system load in a dynamic multi-objective framework. To address the limitation of static weights, the reward weights are adaptively adjusted based on network conditions (e.g., vehicular task density, available RSU bandwidth). Instead of using fixed scalar weights, an adaptive weight adjustment mechanism that modifies the contribution of each objective based on real-time network states and task priorities as depicted by Eq. (22).

$$R_t = w_{p(t)} \cdot R_{\{priority\}} + w_{d(t)} \cdot R_{\{delay\}} + w_{e(t)} \cdot R_{\{energy\}} + w_{l(t)} \cdot R_{\{load\}} \quad (22)$$

Where:

- $R_{\{priority\}}$ rewards tasks with higher application-defined urgency.
- $R_{\{delay\}}$ penalizes end-to-end latency beyond service-level thresholds.
- $R_{\{energy\}}$ penalizes excessive on-board energy consumption during task execution.
- $R_{\{load\}}$ penalizes resource overload at MEC or cloud nodes.

Table 2. Parameters for DRL based Task Offloading

Parameter	Value
State Features	7 (task size, remaining CPU cycles, bandwidth, channel quality, vehicle speed, current load, queue length)
Action Space	{Offload to edge server, Offload to cloud, Execute locally at vehicle }
Hidden Layers	2 fully connected layers (128 neurons, 64 neurons)
Activation Function	ReLU
Output Layer	Linear (Q-values for each action)
Optimizer	Adam
Training Episodes	2000

$w_{p(t)}$, $w_{d(t)}$, $w_{e(t)}$, $w_{l(t)}$ are time-varying weights computed from current system depicted by Eq. (23).

$$w_{i(t)} = \frac{(\text{importance}_{j(t)})}{\sum_j \text{importance}_{i(t)}} \quad (23)$$

The comprehensive details of parameters used for DRL based task offloading is presented by Table 3.

4.2 Proposed algorithms

In this part, we propose an advanced DQN-based algorithm that combines distributed learning, decision-making, and task prioritisation. The intelligent task offloading using ES and CS collaboration is particularly represented by algorithm 1.

The proposed approach is shown in Algorithm 2. This advanced reinforcement learning method has been developed to improve the performance and stability of conventional DQN. Advanced DQN tackles the problem of overestimation in Q-learning by keeping the selection and assessment of actions distinct. Two neural networks that aid in this process are the target and online networks. The online network is responsible for deciding which course of action is best to take in a given situation.

Algorithm 1: Offloading Task Using Base Station

- 1: Vehicle n sends the request for task offloading to the nearest RSU m
- 2: RSU update to BS for task details and resource capacity of the Edge server
- 3: If (task demand) < (Available resource) at ES, then

```

4: Task of n is offloaded to MEC m
Execution server = m
5: Else
6: Select server p using resource optimization,
decision making with the help of BS
7: If the execution server is not m
8: transfer the result to RSU m
9: Provide the final result to the vehicle n
10: else
11: Provide the final result to the vehicle n
12: Update resource capacity of ES to RSU, BS

```

On the other hand, the target network is used to assess the Q-value of the selected action.

In DRL framework, experience replay enhances stability and learning efficiency by breaking temporal correlations and allowing the agent to train on diverse, randomly sampled past experiences. The target network mitigates Q-value oscillations by providing a stable reference for updates, which improves convergence reliability. Together, these mechanisms ensure robust performance and adaptability in highly dynamic vehicular edge computing environments.

The buffer stores previous experiences for subsequent use using a random selection technique.

Algorithm 2: Proposed Algorithm for Task offloading

```

1: Input: S State space, r reward function, A Action
space,
learning rate, replay buffer D, target network
2: Output: optimal task-offloading decision
using resource utilisation
3: Initialize Q-network, D, task priority, agent
4: For episodes 1 through N
5: For each time step, do
6: Select random action with  $\epsilon$  probability
7: If  $\sigma < \epsilon$  then Exploitation
8: select action ( $s_t, a_t$ )
9: else Exploration
10: Select action  $a_t = \operatorname{argmax}_Q(s, a)$ , with max
reward
11: Store  $\langle state, action, reward, newstate \rangle$ 
12: If a sufficient sample in D
13: Compute Q-values, reward
14: Update weight of network
15: Observe optimal Q-network
16: Return: optimizes network with Q-values,
weights

```

5. Performance evaluation

5.1 Simulation settings

We assess a scenario in which five RSUs spread out a 500-meter straight, unidirectional path without any overlap. Each RSU has a 100-meter coverage radius. Vehicles on the road travel between 10 and 50 meters per second. Each zone has between 10 and 50 vehicles. Task sizes, deadlines, Priority were assigned by urgency and resource requirements using Google Cluster Dataset [34]. Python 3.7 and Tensor-Flow 2.4, which provide an effective platform for deploying the proposed approach based on the advanced DQN algorithm in operation and testing it, were used for our simulation study. Table 3 displays the specific VECN parameters.

To ensure that the proposed DRL-based task offloading strategy is evaluated under realistic workload conditions, proposed approach integrates real-world workload traces from the Google Cluster Dataset into vehicular simulation framework. The dataset, collected from a large-scale production cloud cluster, contains over 29 days of job execution records, including task submission times, resource requests (CPU, memory), task priorities, and execution durations. In simulation setup, task submission times were mapped to represent vehicular task generation rates, resource usage metrics were mapped to vehicular computational demands, and task priorities were aligned with heterogeneous QoS requirements in vehicular edge scenarios. This approach preserves realism from real-world vehicular environment while grounding computation demands in production-grade workload data, enabling a robust evaluation of our DRL-based offloading algorithm under conditions closely resembling operational VECNs. Task sizes in experiments ranged from 15 MB to 55 MB for delay and energy evaluation, and 100 MB to 500 MB for failure rate assessment. Network parameters such as available bandwidth and channel conditions were generated based on the statistical distribution observed in the dataset.

The proposed approach incorporates Google Cluster workload traces for computational demand modeling, to simulate realistic vehicle task generation patterns. This integration allows the DRL agent to operate under realistic traffic flow and vehicle's demand variability for resource management, making the simulation environment more representatives of actual VECN conditions. While the Google Cluster Dataset is not a vehicular mobility dataset per se, its workload patterns are

Table 3. VECN Simulation Parameters Setting

Description	Values
Number of vehicles in a zone	10 – 50
Number of edge servers	5
Size of Task	[2 – 20] MB
Deadline of Task	[1 – 5] s
Task Priority	[1 – 10]
Processing capacity of edge servers	100 GHz
Processing capacity of vehicle	20 GHz
Mini-batch size	128
Vehicle velocity	[10 – 50] m/s
RSU's bandwidth	20 MHz
BS's bandwidth	40 MHz
Exploration rate	[1.0 – 0.01]
The discount factor	0.99
Size of the replay buffer	6000
Activation function	ReLU
The learning rate	0.001

highly relevant to vehicular edge computing task scheduling, as they exhibit realistic fluctuations in task arrival rates, execution durations, and resource consumption, closely mirroring the stochastic nature of computation-intensive and latency-sensitive vehicular applications.

A system without optimising techniques for resource allocation and task offloading is represented by the baseline scenario. Without taking into consideration factors like task urgency, the state of the network, or resource availability, tasks are offloaded to edge servers and carried out in this baseline.

5.2 Result and Discussion

This paper evaluates the proposed algorithm's performance in comparison to baseline algorithms, which are local processing, DQN, and Q-learning, by considering task which is generated by vehicles at the vehicle level. These baseline algorithms are compared in term of delay, energy consumption in task completion, and failure rate in task offloading with task size.

- 1) DQN, Conventional DQN techniques assess the choice of actions using a single value function. Overestimation is inevitable because DQN algorithms typically select the course of action that maximises the reward value of the subsequent step;
- 2) Q-learning, traditional temporal difference technique always selects the target as the highest action value at the subsequent instant. It must also keep track of all state-action pairs' rewards;

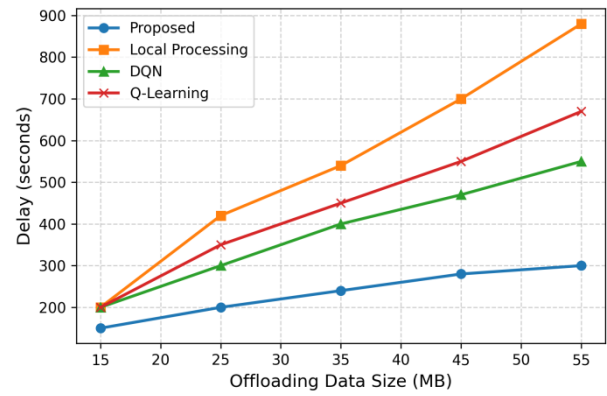


Figure. 3 Delay in task completion with task size (MB)

- 3) Local computing is a baseline approach in which all vehicles use an internal processing unit to carry out their tasks locally.

a) Impact of offloading task size with a delay in completion of task

The impact of offloading data sizes, which range from 10 MB to 60 MB is considered. Fig. 3 shows the relationship between offloading data size and task completion delay for the four evaluated methods. Proposed approach consistently achieves the lowest delay across all task sizes, with performance advantages becoming more pronounced at larger task sizes due to its adaptive resource allocation.

RSUs are able to handle massive volumes of data because to the increasing demand for proximity and preventive services. In conclusion, the suggested method outperforms other current systems in terms of delay. The performance advantage improves as the volume of offloading data increases.

b) Impact of offloading task size on consumption of energy during task completion

Fig. 4, illustrates that proposed approach significantly reduces energy consumption compared to Local Processing, DQN, and Q-learning. The energy-saving benefit is attributed to Proposed's optimized decision-making, which avoids unnecessary long-distance transmissions and reduces CPU load.

The proposed algorithm's effectiveness is demonstrated by the numerical analysis of energy usage for the various algorithms Deep Q-network, Q-Learning, and the proposed algorithm across offloading data sizes ranging from 10 MB to 60 MB.

c) Impact of offloading failure rate with task size

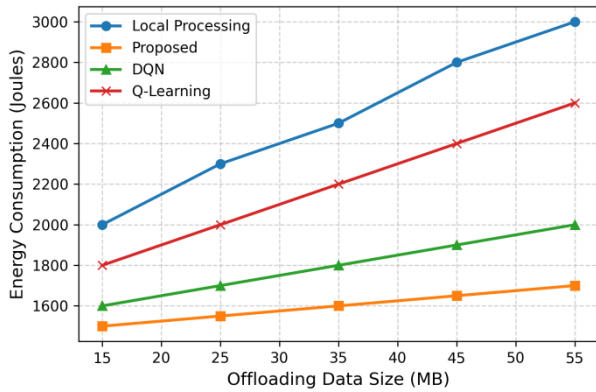


Figure. 4 Energy Consumption with task size (MB)

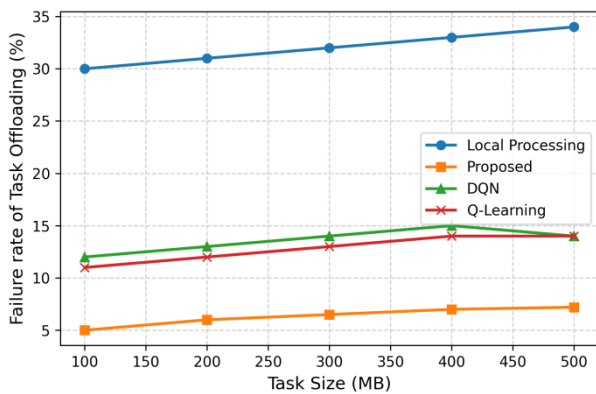


Figure. 5 Task offloading failure rate with task size (MB)

Fig. 5 presents the task failure rate as a function of task size. Proposed approach maintains a failure rate below 5% even for large tasks (500 MB), while other methods, especially Local Processing, show much higher rates due to limited computing capacity. The resilience of the task offloading strategy, which uses task priority, optimal resource allocation, and decision making based on DRL factors that other baseline algorithms ignore, as demonstrated by the lower task offloading failure rate for the proposed approach.

Performance evaluation of proposed DRL-based offloading strategy under varying vehicular task densities and dynamic environment with fluctuating RSU coverage and bandwidth which is considered in computation model is for delay and energy consumption for task completion.

Results show that the proposed approach sustains high offloading success rates and low delays despite dynamic environment, demonstrating strong adaptability to vehicular network.

The proposed DRL-based offloading framework has a per-step training time complexity of $O(d + a + 2 \sum_{l=1}^L n_l n_{l-1})$ and an inference complexity of $O(d + \sum_{l=1}^L n_l n_{l-1})$, where d is the state

dimension, a is the action space, and n_l is the number of neurons in layer l . The space complexity is $O(\sum_{l=1}^L n_l n_{l-1} + M(d+1))$, with M as the replay buffer size. The model's compact architecture ensures real-time decision-making (<10 ms) and feasible memory usage for MEC devices.

5.3 Partial offloading

In the current work, task offloading is formulated as a binary decision (local execution or complete offloading) to maintain tractability and enable efficient convergence of the DRL model in dynamic vehicular environments. However, partial offloading and layered task partitioning can offer finer-grained resource utilization and improved performance in scenarios with heterogeneous task dependencies and variable network conditions. Partial offloading can be incorporated into our framework by extending the action space to represent multiple task segments or hierarchical execution layers. This would require integrating additional state features (e.g., task dependency graphs, intermediate computation costs) and adapting the DRL agent to handle a multi-dimensional action space, which will be explored in our future work.

6. Conclusion and future work

This work introduced proposed approach, a deep reinforcement learning-based task offloading framework for vehicular edge computing. By integrating an enhanced DQN with priority-aware offloading, adaptive reward formulation, and real workload characteristics from the Google Cluster Dataset, Task Offloading using DRL achieves significant improvements in delay, energy consumption, and task success rate. Experimental evaluations against DQN, Q-learning, and Local Processing baselines validate the improved performance of proposed approach, making it a promising solution for latency-sensitive and resource-constrained vehicular applications. Future work plans to implement multi-stage DRL for partial offloading.

Conflicts of interest

The authors confirm the absence of conflicts of interest and the work's compliance with the ethical guidelines.

Author contributions

Shailja Kumari: Conceptualisation, Investigation, Visualisation, Methodology and Original Writing.

Divya Gupta: investigating, writing, editing, and reviewing.

Acknowledgments

The authors have not received any financial support or funding to report

References

- [1] H. Peng and X. Shen, "Deep reinforcement learning based resource management for multi-access edge computing in vehicular networks", *IEEE Trans. Netw. Sci. Eng.*, Vol. 7, No. 4, pp. 2416–2428, 2020.
- [2] B. Saoud *et al.*, "Artificial Intelligence, Internet of things and 6G methodologies in the context of Vehicular Ad-hoc Networks (VANETs): Survey", *ICT Express*, 2024.
- [3] M. M. Rahman and J.-C. Thill, "Impacts of connected and autonomous vehicles on urban transportation and environment: A comprehensive review", *Sustain. Cities Soc.*, Vol. 96, p. 104649, 2023.
- [4] D. Oladimeji, K. Gupta, N. A. Kose, K. Gundogan, L. Ge, and F. Liang, "Smart transportation: an overview of technologies and applications", *Sensors*, Vol. 23, No. 8, p. 3880, 2023.
- [5] M. M. Elsayed, K. M. Hosny, M. M. Fouda, and M. M. Khashaba, "Vehicles communications handover in 5G: A survey", *ICT Express*, Vol. 9, No. 3, pp. 366–378, 2023.
- [6] B. Zhou and R. Buyya, "Augmentation techniques for mobile cloud computing: A taxonomy, survey, and future directions", *ACM Comput. Surv.*, Vol. 51, No. 1, pp. 1–38, 2018.
- [7] J. Zhao, Q. Li, Y. Gong, and K. Zhang, "Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks", *IEEE Trans. Veh. Technol.*, Vol. 68, No. 8, pp. 7944–7956, 2019.
- [8] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, and T. Taleb, "Survey on multi-access edge computing for internet of things realization", *IEEE Commun. Surv. Tutorials*, Vol. 20, No. 4, pp. 2961–2991, 2018.
- [9] G. Guerrero-Contreras, J. L. Garrido, S. Balderas-Diaz, and C. Rodriguez-Dominguez, "A context-aware architecture supporting service availability in mobile cloud computing", *IEEE Trans. Serv. Comput.*, Vol. 10, No. 6, pp. 956–968, 2016.
- [10] S. Liu, L. Liu, J. Tang, B. Yu, Y. Wang, and W. Shi, "Edge computing for autonomous driving: Opportunities and challenges", In: *Proc. IEEE*, Vol. 107, No. 8, pp. 1697–1716, 2019.
- [11] F. Lyu *et al.*, "Service-oriented dynamic resource slicing and optimization for space-air-ground integrated vehicular networks", *IEEE Trans. Intell. Transp. Syst.*, Vol. 23, No. 7, pp. 7469–7483, 2021.
- [12] R. Deng, R. Lu, C. Lai, and T. H. Luan, "Towards power consumption-delay tradeoff by workload allocation in cloud-fog computing", In: *Proc. of 2015 IEEE International Conference on Communications (ICC)*, pp. 3909–3914, 2015, doi: 10.1109/ICC.2015.7248934.
- [13] R. Deng, Z. Yang, J. Chen, N. R. Asr, and M.-Y. Chow, "Residential energy consumption scheduling: A coupled-constraint game approach", *IEEE Trans. Smart Grid*, Vol. 5, No. 3, pp. 1340–1350, 2014.
- [14] B. Yang *et al.*, "Edge intelligence for autonomous driving in 6G wireless system: Design challenges and solutions", *IEEE Wirel. Commun.*, Vol. 28, No. 2, pp. 40–47, 2021.
- [15] C. Li *et al.*, "Dynamic offloading for multiuser multi-CAP MEC networks: A deep reinforcement learning approach", *IEEE Trans. Veh. Technol.*, Vol. 70, No. 3, pp. 2922–2927, 2021.
- [16] Y. Chen, J. Zhao, Y. Wu, J. Huang, and X. Shen, "QoE-Aware Decentralized Task Offloading and Resource Allocation for End-Edge-Cloud Systems: A Game-Theoretical Approach", *IEEE Trans. Mob. Comput.*, Vol. 23, No. 1, pp. 769–784, 2024, doi: 10.1109/TMC.2022.3223119.
- [17] L. Gu, D. Zeng, S. Guo, A. Barnawi, and Y. Xiang, "Cost efficient resource management in fog computing supported medical cyber-physical system", *IEEE Trans. Emerg. Top. Comput.*, Vol. 5, No. 1, pp. 108–119, 2015.
- [18] W. Fan *et al.*, "Joint Task Offloading and Resource Allocation for Vehicular Edge Computing Based on V2I and V2V Modes", *IEEE Trans. Intell. Transp. Syst.*, Vol. 24, No. 4, pp. 4277–4292, 2023, doi: 10.1109/TITS.2022.3230430.
- [19] Y. Liu, H. Yu, S. Xie, and Y. Zhang, "Deep Reinforcement Learning for Offloading and Resource Allocation in Vehicle Edge Computing and Networks", *IEEE Trans. Veh. Technol.*, Vol. 68, No. 11, pp. 11158–11168, 2019, doi: 10.1109/TVT.2019.2935450.
- [20] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks", *IEEE*

- Internet Things J.*, Vol. 6, No. 3, pp. 4377–4387, 2018.
- [21] Y. He, N. Zhao, and H. Yin, “Integrated Networking, Caching, and Computing for Connected Vehicles: A Deep Reinforcement Learning Approach”, *IEEE Trans. Veh. Technol.*, Vol. 67, No. 1, pp. 44–55, 2018, doi: 10.1109/TVT.2017.2760281.
- [22] W. Zhan *et al.*, “Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing”, *IEEE Internet Things J.*, Vol. 7, No. 6, pp. 5449–5465, 2020.
- [23] M.-H. Chen, M. Dong, and B. Liang, “Joint offloading decision and resource allocation for mobile cloud with computing access point”, In: *Proc. of 2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 3516–3520, 2016.
- [24] Y. He, F. R. Yu, N. Zhao, V. C. M. Leung, and H. Yin, “Software-defined networks with mobile edge computing and caching for smart cities: A big data deep reinforcement learning approach”, *IEEE Commun. Mag.*, Vol. 55, No. 12, pp. 31–37, 2017.
- [25] Y. Wang *et al.*, “A game-based computation offloading method in vehicular multiaccess edge computing networks”, *IEEE Internet Things J.*, Vol. 7, No. 6, pp. 4987–4996, 2020.
- [26] S. Li, N. Zhang, H. Chen, S. Lin, O. A. Dobre, and H. Wang, “Joint road side units selection and resource allocation in vehicular edge computing”, *IEEE Trans. Veh. Technol.*, Vol. 70, No. 12, pp. 13190–13204, 2021.
- [27] S. Zafar, S. Jangsher, and A. Zafar, “Federated learning for resource allocation in vehicular edge computing-enabled moving small cell networks”, *Veh. Commun.*, Vol. 45, p. 100695, 2024.
- [28] W. Feng *et al.*, “Energy-efficient collaborative offloading in NOMA-enabled fog computing for Internet of Things”, *IEEE Internet Things J.*, Vol. 9, No. 15, pp. 13794–13807, 2022.
- [29] T. Zhao, S. Zhou, X. Guo, Y. Zhao, and Z. Niu, “A cooperative scheduling scheme of local cloud and internet cloud for delay-aware mobile cloud computing”, In: *Proc. of 2015 IEEE globecom workshops (GC Wkshps)*, pp. 1–6, 2015.
- [30] Y. Dai, D. Xu, S. Maharjan, G. Qiao, and Y. Zhang, “Artificial intelligence empowered edge computing and caching for internet of vehicles”, *IEEE Wirel. Commun.*, Vol. 26, No. 3, pp. 12–18, 2019.
- [31] I. Ullah and Y.-H. Han, “Optimizing vehicular edge computing: graph-based double-DQN approaches for intelligent task offloading”, *J. Supercomput.*, Vol. 81, No. 1, p. 76, 2025.
- [32] Z. Ning, P. Dong, X. Wang, J. J. P. C. Rodrigues, and F. Xia, “Deep reinforcement learning for vehicular edge computing: An intelligent offloading system”, *ACM Trans. Intell. Syst. Technol.*, Vol. 10, No. 6, pp. 1–24, 2019.
- [33] H. Min, A. M. Rahmani, P. Ghaderkourehpaz, K. Moghaddasi, and M. Hosseinzadeh, “A joint optimization of resource allocation management and multi-task offloading in high-mobility vehicular multi-access edge computing networks”, *Ad Hoc Networks*, Vol. 166, p. 103656, 2025.
- [34] H. Yuan, G. Tang, X. Li, D. Guo, L. Luo, and X. Luo, “Online dispatching and fair scheduling of edge computing tasks: A learning-based approach”, *IEEE Internet Things J.*, Vol. 8, No. 19, pp. 14985–14998, 2021.